

Lecture 4

Subtyping

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Motivating Example

New class that can repeatedly generate random numbers:

```
class RandomAnswerGenerator:
    def __init__(self, min_guess: int, max_guess: int):
        # FIXME: Missing input validation
        self.min_guess = min_guess
        self.max_guess = max_guess

    def generate(self) -> int:
        return random.randint(self.min_guess, self.max_guess)
```

Motivating Example

Guessing Game model with flexible answer generation:

```
class GuessingGameModel:
    def __init__(self, min_guess: int, max_guess: int, max_attempts: int,
                  random_answer_generator: RandomAnswerGenerator):
        # ...
        self.random_answer_generator = random_answer_generator
        self.answer = random_answer_generator.generate()

        # ...

    def restart_game(self):
        # ...
        self.answer = self.random_answer_generator.generate()
```

Motivating Example

- What should we change if we want to be able to *reuse our model* for a number guessing game that generates *only odd numbers* as answers?

Motivating Example

- New class that can repeatedly generate odd random numbers:

```
class OddRandomAnswerGenerator:
    def __init__(self, min_guess: int, max_guess: int):
        # ...

    def generate(self) -> int:
        # Self-check: Improve this implementation
        while True:
            answer = random.randint(self.min_guess, self.max_guess)
            if answer % 2 != 0:
                return answer
```

Motivating Example

- How do we make the model depend on an instance of `OddRandomAnswerGenerator` while still allowing `RandomAnswerGenerator` to be used without Pyright complaining?

Subtyping

- **Subtyping:** Hierarchical substitutability or assignability of types
 - Also called *subtype polymorphism* (one type name represents many others)

Subtyping

- If S is a subtype of T , then S values can be used where T values are needed

Notation	Meaning	Implication
$S <: T$ or $S \leq T$	S is a subtype of T	T is a supertype of S
$T <: S$ or $T \leq S$	T is a subtype of S	S is a supertype of T
$S \equiv T$	S is equivalent to T	$S <: T$ and $T <: S$

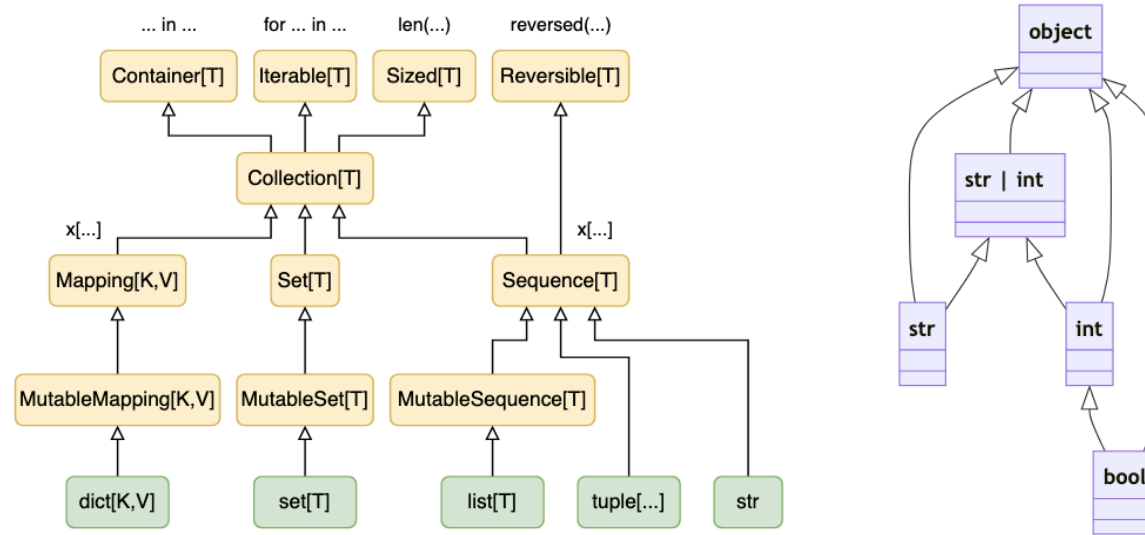
Example in Python: `bool <: int`

- “Booleans can be used in place of integers”; how?

Python Subtyping Hierarchy

- $<:$ is a *preorder relation*
 - *reflexive* ($A <: A$)
 - *transitive* (if $A <: B$ and $B <: C$, then $A <: C$)
 - results in a *hierarchy* of types
- `bool <: int`
- $A <: A \mid B$ for any A and B
- $B <: A \mid B$ for any A and B
- $A <: \text{object}$ for any A

Python Subtyping Hierarchy



- Self-arrows (i.e., `A <: A`) in diagram are omitted for simplicity
- Arrow in diagram denotes “source is a subtype of target”

Back to Motivating Example

- How do we make `__init__` able to take in either `RandomAnswerGenerator` or `OddRandomAnswerGenerator`?

```
class NumberGuessingGameModel:
    def __init__(self, min_guess: int, max_guess: int,
                  max_attempts: int,
                  random_answer_generator: RandomAnswerGenerator):
        # ...
        self.random_answer_generator = random_answer_generator
        self.answer = random_answer_generator.generate()
```

Back to Motivating Example

- Possible answer: Use |:

```
class NumberGuessingGameModel:
    def __init__(self, min_guess: int, max_guess: int,
                  max_attempts: int,
                  random_answer_generator: RandomAnswerGenerator |
                  OddRandomAnswerGenerator):
        # ...
        self.random_answer_generator = random_answer_generator
        self.answer = random_answer_generator.generate()
```

What problems does this approach have?

Interface

- **Interface:** Abstraction for set of properties and method signatures
 - Abstracts out implementation details
 - Interface is “implemented” by concrete classes (interface should not be instantiated)
- *Explicit interface:* `abc.ABC` and `@abstractmethod` in Python (will not be discussed yet)

Interface

- *Implicit interface*: Protocol in Python

```
from typing import Protocol

class A(Protocol):
    def f(self, x: int) -> int:
        # A implementors must have body for f and g
        # f has no method body in A (only signature)
        # Same goes for g

    def g(self):

class B:
    def f(self, x: int) -> int:
        # Implicitly B <: A via Protocol;
        # B has compatible structure with A

    def g(self):
        # f and g of B must have same
        # signatures as those of A
```

Back to Motivating Example

- Another answer: Define a Protocol called AnswerGenerator:

04/gg_protocol

How does this address the issues of the
RandomAnswerGenerator | OddRandomAnswerGenerator
approach?

Self-check

- Use subtyping to refactor the functions below into a single `concat`:

```
def concat_strlist_with_commas(strs: list[str]):  
    return ', '.join(strs)  
  
def concat_chars_with_commas(strs: str):  
    return ', '.join(strs)  
  
def concat_strtuple2_with_commas(strs: tuple[str, str]):  
    return ', '.join(strs)
```

Self-check

```
def concat_strtuple3_with_commas(strs: tuple[str, str, str]):  
    return ','.join(strs)
```

```
# Note 1: Cannot handle `str` tuples of arbitrary length;  
# ensure that your `concat` handles this
```

```
# Note 2: Think of at least two solutions; which is better?
```

Examples of Python subtyping hierarchies:

- abstract base classes
- exception hierarchy
- numeric tower